RELATÓRIO TÉCNICO

Grafana + Prometheus

Monitoramento, métricas, dashboards e observabilidade

Preparado para estudo e apresentação
Outubro de 2026

1. Resumo executivo

Grafana e Prometheus são ferramentas amplamente utilizadas para monitoramento e observabilidade. O Prometheus é responsável principalmente pela coleta e armazenamento de métricas, enquanto o Grafana é utilizado para consultar, visualizar e apresentar essas informações em dashboards e alertas. A combinação permite acompanhar o funcionamento de servidores, APIs, aplicações, containers, bancos de dados e dispositivos IoT, facilitando a identificação de problemas de desempenho e disponibilidade.

2. O que são Grafana e Prometheus?

2.1 Prometheus

Prometheus é uma plataforma de monitoramento baseada em métricas e séries temporais. Ele coleta informações de sistemas e aplicações em intervalos definidos, armazena essas métricas e permite consultá-las por meio da linguagem PromQL.

2.2 Grafana

Grafana é uma plataforma de visualização e observabilidade. Ela pode se conectar ao Prometheus e a diversas outras fontes de dados para transformar métricas em gráficos, tabelas, indicadores, dashboards e alertas.

3. Para que servem?

A principal finalidade da arquitetura Grafana + Prometheus é permitir que uma equipe acompanhe a saúde, disponibilidade e desempenho de seus sistemas.

- Monitorar uso de CPU, memória e armazenamento.
- Acompanhar tráfego e utilização de rede.
- Monitorar APIs e serviços.
- Medir quantidade de requisições e erros.
- Acompanhar tempo de resposta de aplicações.
- Monitorar containers e infraestrutura.
- Monitorar dispositivos IoT e servidores.
- Criar alertas quando uma métrica atingir determinado limite.

4. Como funciona a arquitetura

De forma simplificada, a arquitetura funciona em três etapas: uma aplicação ou equipamento disponibiliza métricas; o Prometheus coleta e armazena essas métricas; e o Grafana consulta os dados para apresentar informações de forma visual.

Aplicação / Servidor / IoT

↓

Métricas / Exporters

↓

Prometheus — coleta e armazena

↓

Grafana — consulta, dashboards e alertas

5. Exemplos de utilização

5.1 Monitoramento de uma API Spring Boot

Uma API desenvolvida em Spring Boot pode disponibilizar métricas relacionadas ao número de requisições, tempo de resposta, utilização de recursos e erros. O Prometheus coleta essas informações e o Grafana apresenta um dashboard com a situação da aplicação.

- Exemplo de indicadores:
  - CPU: 72%
  - Memória: 65%
  - Requisições: 1.250 por minuto
  - Erros HTTP: 12
  - Tempo médio de resposta: 180 ms
  - Status da API: Online

5.2 Monitoramento de Raspberry Pi e IoT

Em um projeto com Raspberry Pi, como um sistema de controle de ponto ou outro equipamento IoT, a arquitetura pode acompanhar CPU, memória, temperatura, armazenamento, disponibilidade do equipamento e outras métricas disponibilizadas pelo sistema ou pela aplicação.

6. Aplicação em um sistema de controle de ponto

Em um sistema como o EduPonto, Grafana e Prometheus poderiam ser utilizados para monitorar a infraestrutura sem interferir diretamente no funcionamento principal do sistema.

- Raspberry Pi online ou offline.
- Uso de CPU e memória.
- Temperatura do equipamento.
- Espaço disponível no armazenamento.
- Estado da comunicação com o servidor.
- Quantidade de registros de ponto processados.
- Quantidade de erros de comunicação.
- Disponibilidade dos serviços.

7. Vantagens

- Permite visualizar o comportamento dos sistemas de forma centralizada.
- Facilita a identificação de falhas e gargalos.
- Permite acompanhar métricas em tempo real ou histórico.

- Possibilita criação de dashboards personalizados.
- Permite configurar alertas para situações críticas.
- Prometheus e Grafana possuem forte adoção na área de infraestrutura e cloud.
- Podem ser utilizados em ambientes com aplicações, servidores, containers e IoT.

8. Limitações e pontos de atenção

A combinação não elimina a necessidade de outras ferramentas de observabilidade. Em ambientes maiores, pode ser necessário complementar métricas com logs e rastreamento distribuído (traces). Também é necessário planejar corretamente retenção de dados, segurança, cardinalidade das métricas e regras de alerta.

9. Principais concorrentes e alternativas

Ferramenta	Principal finalidade	Diferencial
Zabbix	Monitoramento de infraestrutura	Forte foco em servidores, redes e infraestrutura tradicional.
Datadog	Observabilidade	Integra métricas, logs, traces e monitoramento em uma plataforma comercial.
New Relic	Observabilidade de aplicações	Foco forte em desempenho e monitoramento de aplicações.
InfluxDB + Telegraf	Métricas e séries temporais	Muito utilizado em métricas e cenários de IoT.
Nagios	Monitoramento de infraestrutura	Ferramenta tradicional para disponibilidade e infraestrutura.

10. Comparação resumida

Ferramenta	Função principal	Relação com Grafana + Prometheus
Prometheus	Coleta e armazenamento de métricas	É o componente de coleta/armazenamento da arquitetura.
Grafana	Visualização e dashboards	É o componente de visualização da arquitetura.
Zabbix	Monitoramento de infraestrutura	Alternativa integrada para monitoramento.
Datadog	Observabilidade completa	Alternativa comercial integrada.
New Relic	APM e observabilidade	Alternativa focada em aplicações.
InfluxDB	Banco de séries temporais	Alternativa para armazenamento de métricas.
Nagios	Monitoramento de infraestrutura	Alternativa tradicional.

11. Conclusão

Grafana e Prometheus formam uma arquitetura eficiente para monitoramento baseado em métricas. O Prometheus concentra a coleta e o armazenamento das informações, enquanto o Grafana transforma esses dados em visualizações que facilitam a análise. A solução pode ser aplicada desde uma API ou servidor até ambientes com containers e dispositivos IoT. Entre as principais alternativas estão Zabbix, Datadog, New Relic, InfluxDB e Nagios. A escolha depende do nível de monitoramento necessário, do ambiente e do orçamento disponível.

12. Resumo para apresentação

Prometheus coleta e armazena métricas de sistemas e aplicações. Grafana utiliza essas métricas para criar dashboards, gráficos e alertas. Juntos, permitem acompanhar desempenho, disponibilidade e falhas em servidores, APIs, aplicações, containers e dispositivos IoT. Seus principais concorrentes e alternativas incluem Zabbix, Datadog, New Relic, InfluxDB e Nagios.

13. Referências para consulta

- Documentação oficial do Prometheus — https://prometheus.io/docs/
- Documentação oficial do Grafana — https://grafana.com/docs/
- Documentação oficial do Zabbix — https://www.zabbix.com/documentation/
- Documentação oficial do Datadog — https://docs.datadoghq.com/
- Documentação oficial do New Relic — https://docs.newrelic.com/
- Documentação oficial do InfluxDB — https://docs.influxdata.com/
- Documentação oficial do Nagios — https://www.nagios.org/documentation/

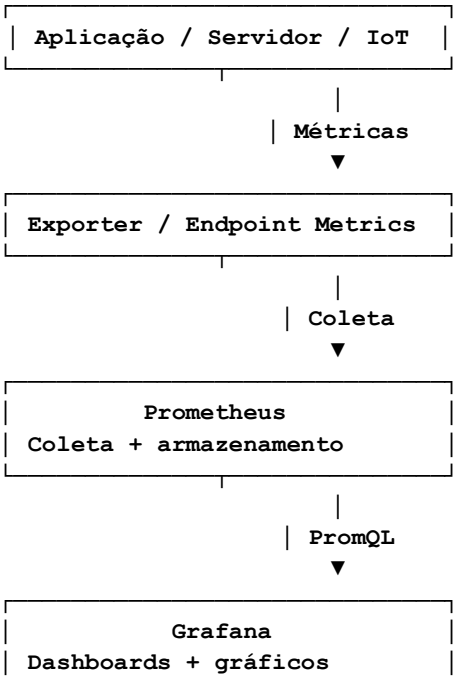
14. Arquitetura Grafana + Prometheus

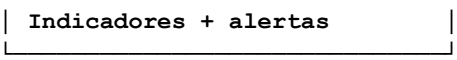
A arquitetura Grafana + Prometheus é baseada na separação entre coleta, armazenamento, consulta e visualização das métricas. O Prometheus atua como o núcleo de coleta e armazenamento de métricas, enquanto o Grafana utiliza essas informações para criar dashboards e alertas.

14.1 Componentes da arquitetura

Componente	Função
Aplicação ou dispositivo	Gera as métricas que representam o funcionamento do sistema. Pode ser uma API, servidor, container, Raspberry Pi ou dispositivo IoT.
Exporter / endpoint de métricas	Disponibiliza métricas em um formato que o Prometheus consegue coletar. Um exemplo comum é o Node Exporter para métricas de servidores.
Prometheus	Consulta periodicamente os endpoints de métricas, coleta os valores e os armazena como séries temporais.
PromQL	Linguagem utilizada para consultar e transformar as métricas armazenadas no Prometheus.
Grafana	Conecta-se ao Prometheus como fonte de dados e apresenta as métricas em dashboards, gráficos, tabelas e indicadores.
Alertas	Permitem identificar situações críticas, como CPU elevada, serviço indisponível ou quantidade excessiva de erros.

14.2 Fluxo da arquitetura

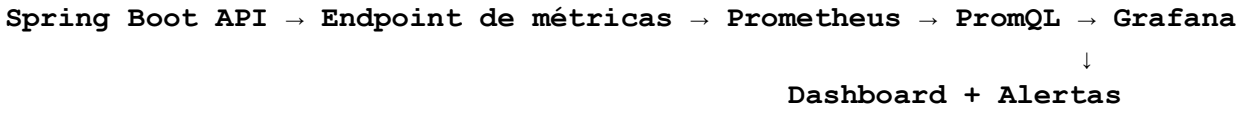




O fluxo começa no sistema monitorado. As métricas são disponibilizadas por um endpoint ou exporter. O Prometheus realiza a coleta e mantém os valores históricos. O Grafana consulta o Prometheus utilizando PromQL e transforma os resultados em informações visuais.

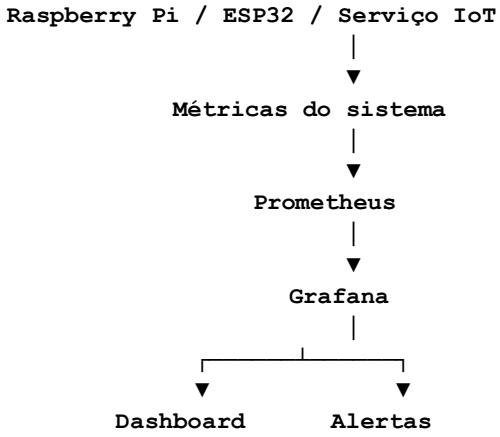
14.3 Arquitetura aplicada a uma API Spring Boot

Em uma API Spring Boot, a aplicação pode disponibilizar métricas sobre requisições, erros, tempo de resposta, memória e outros indicadores. O Prometheus coleta essas informações e o Grafana apresenta um painel para acompanhar o estado da API.



14.4 Arquitetura aplicada a Raspberry Pi e IoT

Em um ambiente IoT, o Raspberry Pi ou outro dispositivo pode fornecer métricas de infraestrutura e da própria aplicação. Essas informações podem ser enviadas para o ambiente de monitoramento, permitindo acompanhar disponibilidade, CPU, memória, temperatura, armazenamento e eventos.



14.5 Vantagens dessa arquitetura

- Separação clara entre coleta de métricas e visualização.
- Possibilidade de centralizar métricas de vários servidores e aplicações.
- Dashboards personalizados para diferentes equipes ou sistemas.
- Consultas utilizando PromQL.
- Monitoramento histórico e acompanhamento de tendências.
- Possibilidade de criação de alertas para eventos críticos.
- Boa adaptação a ambientes com APIs, containers, servidores e IoT.

15. Observação final sobre a arquitetura

A arquitetura Grafana + Prometheus não significa que Grafana e Prometheus executam a mesma função. O Prometheus é responsável principalmente pelas métricas e pelo armazenamento de

séries temporais, enquanto o Grafana atua principalmente na consulta, visualização e apresentação dessas informações. Essa separação permite que os componentes sejam utilizados de maneira independente ou combinados em uma solução de observabilidade.